Computer Programming Assignment

Submitted by : Harshit Kakkar
Roll Number : 12611043
Branch : CSE A
Email :
harshitcse12611043@iiitsonepat.ac.in

Submitted On 22 October 2026
Course : BTech 2026-2030

Submitted To: Dr. Gourav Jain

1. Write a C program to input two integers and display their sum, difference, product, quotient, and remainder. If the second number is zero, display an appropriate message instead of performing division or modulus.

Answer:

```c
#include <stdio.h>
int get_int(char* prompt);
int main()
{
    int a = get_int("Enter the first number a: ");
    int b= get_int("Enter the second number b: ");
    // checking if the second number is zero or not


    printf("The sum of the two numbers is %i\n",a+b);
    printf("The difference of the two numbers a-b is %i and b-a is
%i\n", a-b, b-a);
    printf("The product of the two numbers is %i\n",a*b);
    if(b==0)
    {
        printf("The quotient a/b is not possible because the second
number is zero");

    }
    else
    {
        printf("The quotient of a/b is %0.2f\n",a/(float)b);
        printf("The remainder of the division a/b is %d\n", a%b);
    }


}
int get_int(char* prompt)
{
    int a;
    printf("%s\n",prompt);
    scanf("%d",&a);
    return a;
}
```

Output:

```
Enter the first number a:
5
Enter the second number b:
8
The sum of the two numbers is 13
The difference of the two numbers a-b is -3 and b-a is 3
The product of the two numbers is 40
The quotient of a/b is 0.62
The remainder of the division a/b is 5
```

2. Write a C program to input three integers and find the largest and smallest number using conditional statements.

Answer :

```c
#include <stdio.h>
int smallest(int a, int b, int c);
int greatest(int a, int b, int c);
int get_int(char* prompt);
int main(void)
{
    int a = get_int("Enter the first number: ");
    int b = get_int("Enter the second number: ");
    int c = get_int("Enter the third number: ");
    if(a==b && b==c)
    {
        printf("The three numbers are equal\n");
        return 0;
    }
    printf("Greatest of these three numbers is %d\n",
greatest(a,b,c));
    printf("Smallest of these three numbers is
%d\n",smallest(a,b,c));
    return 0;
}

int smallest(int a, int b, int c)
{
    if(a<b)
    {
        if(a<c)
            return a;
        else
            return c;
    }
```

```
    else
    {
        if(b<c)
            return b;
        else
            return c;
    }

}
int greatest(int a, int b, int c)
{
    if(a>b)
    {
        if (a>c)
        {
            return a;
        }
        else
        {
            return c;
        }
    }
    else
    {
        if(b>c)
        {
            return b;
        }
        else {
            return c;
        }
    }
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
```

Sample Output:

```
Enter the first number: 6
Enter the second number: 90
Enter the third number: 78
Greatest of these three numbers is 90
Smallest of these three numbers is 6
```

3.Write a C program to input the marks of five subjects. Calculate the total and percentage and display the grade according to the following criteria: 90 and above – A+, 80–89 – A, 70–79 – B, 60–69 – C, 50–59 – D, below 50 – F. Display Invalid Marks if any mark is outside 0–100.

```c
#include <stdio.h>
char grade(float arg);
int get_int(char* prompt);
int check(int a);

int main()
{
    printf("There are five subjects: Maths, Chemistry, Hindi, Social Science and Science\n");
    printf("Enter each of the marks one by one out of hundred\n");
    int a = get_int("Enter the marks of Maths: ");

    if(check(a))
        return 1;
    int b = get_int("Enter the marks of Chemistry: ");

    if(check(b))
        return 1;
    int c = get_int("Enter the marks of Hindi: ");

    if(check(c))
        return 1;
    int d = get_int("Enter the marks of Social Science: ");

    if(check(d))
        return 1;
    int e = get_int("Enter the marks of Science: ");

    if(check(e))
        return 1;
    // printing the total marks out of 500
    printf("The total marks are %d out of 500\n",(a+b+c+d+e));
    //calculating percentage
    float percent=(a+b+c+d+e)*0.2;
    printf("The percentage is %0.2f\n", percent);
    printf("The grade is %c", grade(percent));
    if(percent >= 90)
    {
        printf("+");
    }

    printf("\n");
    return 0;

}
```

```c
char grade(float arg)

{
    //assigining grades
    if(arg>=80 && arg<=100)
    {
        return 'A';
    }
    else if(arg>=70 && arg<80)
    {
        return 'B';
    }
    else if(arg>=60 && arg<70)
    {
        return 'C';

    }
    else if(arg>=50 && arg<60)
    {
        return 'D';
    }
    else
    {
        return 'F';
    }

}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
int check(int a)
{
    // checking if the given marks are within the allowed range
    if (a<0||a>100)
    {
        printf("INVALID MARKS, PLEASE START AGAIN\n");
        return 1;
    }
    return 0;
}
```

Sample Output 1

```
There are five subjects: Maths, Chemistry, Hindi, Social Science and
Science
Enter each of the marks one by one out of hundred
Enter the marks of Maths: 98
Enter the marks of Chemistry: 90
Enter the marks of Hindi: 97
Enter the marks of Social Science: 99
Enter the marks of Science: 90
The total marks are 474 out of 500
The percentage is 94.80
The grade is A+
```

Sample Output 2

```
There are five subjects: Maths, Chemistry, Hindi, Social Science and
Science
Enter each of the marks one by one out of hundred
Enter the marks of Maths: 45
Enter the marks of Chemistry: 67
Enter the marks of Hindi: 311
INVALID MARKS, PLEASE START AGAIN
```

4. Write a C program to calculate an electricity bill. The first 100 units are charged at ₹2 per unit, the next 100 units at ₹3 per unit, the next 200 units at ₹5 per unit, and units above 400 at ₹7 per unit. If the final bill exceeds ₹2000, add a surcharge of ₹100.

Answer:

```c
#include <stdio.h>
int get_int(char* prompt);
int main(void)
{
    int a = get_int("Enter the units used this month: ");
    char b;
    long bill =0;
    // The first 100 units are charged at ₹2 per unit, the next 100 units at ₹3 per unit, the next 200 units at ₹5 per unit, and units above 400 at ₹7 per unit.
    if(a>400)
    {
        b = 'A';
        bill = (a-400)*7 + 1500;
    }
    else if(a>200 && a<=400)
    {
        b = 'B';
        bill = (a-200)*5 + 500;
    }
    else if(a>100 && a<=200)
    {
        b = 'C';
        bill = (a-100)*3 + 200;
    }
    else
    {
        b ='D';
        bill = a * 2;
    }
    // If the final bill exceeds ₹2000, add a surcharge of ₹100.
    if(bill>2000)
        bill += 100;
    printf("The Electricity bill for this month is %li\n", bill);
    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
```

Sample Output :

```
Enter the units used this month: 1545
The Electricity bill for this month is 9615
```

5.Write a C program to input a positive integer and print its reverse, the sum of its digits, the number of digits, and the largest digit using loops.
Answer:

```
#include <stdio.h>
int get_int(char* prompt);
int digitCount(long int x);
int reverse(int a);
int digitSum(int num );
int largestDigit(int num);
int main(void)
{
    int num= get_int("Enter the number");
    // reverse(num);
    printf("The reverse of the number is: %i\n", reverse(num));
    // digitSum(num)
    printf("The sum of the digits of the number is: %i\n",
digitSum(num));
    //digitCount(num)
    printf("The number of the digits of the number is: %i\n",
digitCount(num));
    // largestDigit(num)
    printf("The largest of the digits of the number is: %i\n",
largestDigit(num));
    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
int digitCount(long int x)
{
    //counting digits
    if (x == 0)
        return 1;
    if (x < 0)
        x = -x;
    int count = 0;
    while(x>0)
    {
        x=x/10;
        count++;
    }
```

```
    return count;
}
int reverse(int a)
{
    int res=0;
    int count = digitCount(a);
    // starting from the right end and adding the digits by
multiplying the already number by 10;
    for(int i =0; i<count; i++)
    {
        int digit = a%10;
        a = (a/10);
        res = res*10 + digit;

    }

    return res;
}
int digitSum(int a)
{
    int res=0;
    int count = digitCount(a);
    //adding the digits starting from units place
    for(int i =0; i<count; i++)
    {
        int digit = a%10;
        a = (a/10);
        res += digit;

    }
    return res;
}
int largestDigit(int num)
{
    //finding the largest digit out of the ones in the given number
    int greatest = num%10;
    int count = digitCount(num);
    for(int i =0; i<count; i++)
    {
        int digit = num%10;
        num = (num/10);
        if(digit>greatest)
        {
            greatest = digit;
        }

    }
    return greatest;
}
```

```
Enter the number 78654
The reverse of the number is: 45687
The sum of the digits of the number is: 30
The number of the digits of the number is: 5
The largest of the digits of the number is: 8
```

6. Write a C program to input two positive integers and find their GCD and LCM. Implement the solution using loops without using any built-in GCD/LCM function.

Answer:

```c
#include <stdio.h>
int get_int(char* prompt);
int gcd(int a, int b);
int lcm(int a, int b);
int main(void)
{
    int a = get_int("Enter first positive integer: ");
    int b = get_int("Enter second positive integer: ");
    int g = gcd(a, b);
    printf("The GCD of %d and %d is %d\n", a, b, g);
    int l = lcm(a, b);
    printf("The LCM of %d and %d is %d\n", a, b, l);

    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}

int gcd(int a, int b)
{
    // using euclidean algorithm to find GCD
    if(b>a) //swapping the numbers to keep b>a
    {
        int t = b;
        b = a;
        a = t;
    }
    while(a != 0)
    {
        int t = b;
        b = a%b;
        a = t;

    }
    return b;
}

int lcm(int a, int b)
{
```

```
    //if any of the numbers is 0 return 0 immediately
    if(a==0 || b==0)
    {
        return 0;
    }
    //swapping the numbers to keep b>a
    if(b>a)
    {
        int t = b;
        b = a;
        a = t;
    }
    //first common multiple is the LCM
    int i = a;
    while(i%b != 0)
    {
        a++;
    }

    return a-1;
}
```

Output

```
Enter first positive integer: 56
Enter second positive integer: 64
The GCD of 56 and 64 is 8
The LCM of 56 and 64 is 448
```

7.Write a C program to input a positive integer and determine whether it is a prime number. Then print all prime numbers from 2 up to the given number. Use a user-defined function isPrime() for checking primality.
Answer:

```c
#include <stdio.h>
int get_int(char* prompt);
int isPrime(int a);

int main(void)
{
    int a = get_int("Enter the positive integer: ");
    if(isPrime(a))
    {
        printf("The number is prime.\n");
    }
    else
    {
        printf("The number is not prime.\n");
    }
    printf("The prime numbers from 2 to %d are:\n", a);
    for (int i = 2; i<=a; i++)
    {
        if(isPrime(i))
        {
            printf("%d\t", i);
        }
    }
    printf("\n");
    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}

int isPrime(int a)
{
    int count = 0;
    for (int i = 1; i<=a; i++)
    {
        if(a%i==0)
        {
            count++;
        }
    }
    if(count==2)
    {
        return 1;
    }
```

```
    else
    {
        return 0;
    }
}
```

Output 1

```
Enter the positive integer: 56
The number is not prime.
The prime numbers from 2 to 56 are:
2       3       5       7       11      13      17      19      23
29      31      37      41      43      47      53
```

Output 2

```
Enter the positive integer: 23
The number is prime.
The prime numbers from 2 to 23 are:
2       3       5       7       11      13      17      19      23
```

8. Write a C program to input a number and determine whether it is a palindrome, Armstrong number, Strong number, or Perfect number. Implement each check using a separate user-defined function.

```
#include <stdio.h>
#include <math.h>
int get_int(char* prompt);
int dcount(int a);
long fac(int a);
int isPalindrome(int a);
int isPerfect(int a);
int isStrong(int a);
int isArmstrong(int a);

int main(void)
{
    int a;
    do
    {
        a = get_int("Please enter a number: ");

    }
    while(a<0);
    if(isPalindrome(a)) printf("The given number is a palindrome\n");
    else printf("The given number is not a palindrome\n");
    if(isStrong(a)) printf("The given number is a Strong number\n");
    else printf("The given number is not a Strong number\n");
    if(isArmstrong(a)) printf("The given number is a armstrong
number\n");
    else printf("The given number is not a armstrong number\n");
```

```
     if(isPerfect(a)) printf("The given number is a perfect
number\n");
    else printf("The given number is not a perfect number\n");
    return 0;
}
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
}
int dcount(int a)
{
    if(a==0) return 1;
    int count= 0;
    while(a>0)
    {
        a=a/10;
        count++;
    }
    return count;
}
long fac(int a)
{
    if(a == 1 || a==0) return 1;
    return a*fac(a-1);
}
int isPalindrome(int a)
{
    int reverse = 0;
    int copy = a;
    while(a>0)
    {
        int digit = a%10;
        a=a/10;
        reverse = (reverse*10) + digit;
    }
    if(copy == reverse)
    {
        return 1;
    }
    return 0;
}
int isPerfect(int a)
{
    int sum=0;
    for(int i = 1; i<a; i++)
    {
        if (a%i == 0) sum+=i;

    }

    if(sum==a)
    {
        return 1;
    }
    return 0;
```

```
int isStrong(int a)
{
    int sum=0;
    int count = dcount(a);
    int copy = a;
    for(int i = 0; i<count; i++)
    {
        sum += fac(a%10);
        a/=10;
    }
    if (sum==copy){
        return 1;
    }
    return 0;
}
int isArmstrong(int a)
{
    int power = dcount(a);
    int sum = 0;
    int copy = a;
    for(int i =0; i<power; i++)
    {
        sum+= pow(a%10, power);
        a/=10;
    }
    if(sum == copy) return 1;
    return 0;
}
```

Sample Output:

```
Please enter a number: 6
The given number is a palindrome
The given number is not a Strong number
The given number is a armstrong number
The given number is a perfect number
```

9.Write a C program to input two integers L and R and print all Armstrong numbers between L and R. Create separate functions for counting the digits, calculating a power, and checking whether a number is an Armstrong number.
Answer:

```c
#include <stdio.h>
int get_int(char* prompt);
int isArmstrong(int a);
int dcount(int a);
int getPower(int a, int b);


int main(void)
{
    int L = get_int("Enter the lower limit: ");
    int R = get_int("Enter the upper limit: ");
    int counter = 0;
    printf("The armstrong numbers between L and R\n");
    for(int i = L+1; i<R; i++)
    {
        if(isArmstrong(i))
        {
            counter++;
            printf("%d\t", i);
        }

    }
    if(counter==0) printf("There are no armstrong numbers between L
and R");
    printf("\n");
    return 0;
}
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
}
int dcount(int a)
{
    if(a==0) return 1;
    int count= 0;
    while(a>0)
    {
        a=a/10;
        count++;
    }
    return count;
}
```

```
int getPower(int a, int b)
{
    int ans =1;
    for(int i = 0; i < b; i++) ans *=a;
    return ans;
}
int isArmstrong(int a)
{
    int power = dcount(a);
    int sum = 0;
    int copy = a;
    for(int i =0; i<power; i++)
    {
        sum+= getPower(a%10, power);
        a/=10;
    }
    if(sum == copy) return 1;
    return 0;
}
```

Sample Output:

```
Enter the lower limit: 100
Enter the upper limit: 500
The armstrong numbers between L and R
153     370     371     407
```

10. Write a C program to print the following pattern for n rows. The numbers in the pattern must be
consecutive prime numbers.

```c
#include <stdio.h>
int get_int(char* prompt);
int isPrime(int a);
int main(void)
{
    int a = get_int("Enter a positive integer: ");
    //number of prime numbers needed
    int n = (a*(a+1))/2;
    int j = 1;
    for(int i = 1;i<=a; i++)
    {
        int count =0;
        while(count != i )
        {
            if(isPrime(j))
            {
                printf("%d ", j);
                count++;
            }
            j++;
        }
        printf("\n");
    }
    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
int isPrime(int a)
{
    int count = 0;
    for (int i = 1; i<=a; i++)
    {
        if(a%i == 0)
        {
            count ++;
        }
    }
    if(count == 2)
    {
        return 1;
    }
    return 0;
}
```

For n = 4:
2
3 5
7 11 13
17 19 23 29
Use a user-defined isPrime() function.
Sample Output:

```
Enter a positive integer: 4
2
3 5
7 11 13
17 19 23 29
```

11.Write a C program to input a positive integer and find the sum of its digits, product of its digits, largest digit, smallest digit, number of even digits, and number of odd digits. Implement the required operations using user-defined functions.

```c
#include <stdio.h>
int get_int(char* prompt);
long long sumDigits(int a);
long long productDigits(int a);
int largestDigit(int a);
int smallestDigit(int a);
int evenDigit(int a);
int oddDigit(int a);

int main(void)
{
    int a;
    do{
        a = get_int("Please enter a positive integer: ");

    }while(a<0);
    printf("The sum of digits is : %lli\n",sumDigits(a));
    printf("The product of digits is : %lli\n",productDigits(a));
    printf("The largest of digits is : %d\n",largestDigit(a));
    printf("The smallest of digits is : %d\n",smallestDigit(a));
    printf("The number of even digits is : %d\n",evenDigit(a));
    printf("The number of odd digits is : %d\n",oddDigit(a));
    return 0;
}
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
```

```
}
long long sumDigits(int a)
{
    long long sum = 0;
    while(a>0)
    {
        sum += a%10;
        a/=10;
    }
    return sum;
}
long long productDigits(int a)
{
    long long product = 1;
    while(a>0)
    {
        product *= a%10;
        a/=10;
    }
    return product;
}
int largestDigit(int a)
{
    int greatest = a%10;
    while(a>0)
    {
        if(greatest<a%10)
        {
            greatest = a%10;
        }
        a/=10;
    }
    return greatest;
}
int smallestDigit(int a)
{
    int smallest = a%10;
    while(a>0)
    {
        if(smallest>a%10)
        {
            smallest = a%10;
        }
        a/=10;
    }
    return smallest;
}
int evenDigit(int a)
{
    int even = 0;
    while(a>0)
    {
        if((a%10)%2 == 0)
        {
            even++;
        }
        a/=10;
    }
```

```c
int oddDigit(int a)
{
    int odd = 0;
    while(a>0)
    {
        if((a%10)%2 != 0)
        {
            odd++;
        }
        a/=10;
    }
    return odd;
}
```

Output:

```
Please enter a positive integer: 123456
The sum of digits is : 21
The product of digits is : 720
The largest of digits is : 6
The smallest of digits is : 1
The number of even digits is : 3
The number of odd digits is : 3
```

12. Write a C program to input a positive integer and determine whether it is an Automorphic Number. A number is Automorphic if its square ends with the same digits as the number itself. For example, $25^2 = 625$, so 25 is an Automorphic Number. Implement the check using a function.

```c
#include <stdio.h>
int get_int(char* prompt);
int dcount(int a);
long getPower(int a, int b);
long isAutomorphic(int a);

int main(void)
{
    int a;
    do{
        a = get_int("Enter a positive integer: ");

    }while(a <0);
    if (isAutomorphic(a))
    {
        printf("The given number is an automorphic number.\n");
        return 0;
    }
    printf("The given number is not an automorphic number\n");
    return 1;

}
```

```c
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
}
int dcount(int a)
{
    int count = 0;
    while(a>0)
    {
        a/=10;
        count++;
    }
    return count;
}
long getPower(int a, int b)
{
    long ans = 1;
    for(int i = 0; i<b; i++)
    {
        ans*=a;
    }
    return ans;
}
long isAutomorphic(int a)
{
    long sq = a*a;
    int c = dcount(a);
    if((sq%(getPower(10,c)))==a) return 1;
    return 0;
}
```

Output:

```
Enter a positive integer: 45
The given number is not an automorphic number

Enter a positive integer: 5
The given number is an automorphic number.
```

13.Write a C program to input two integers L and R and display all Perfect Numbers in that range. A Perfect Number is a number whose sum of proper positive divisors is equal to the number itself. Also display the total number of Perfect Numbers found. Use a user-defined function isPerfect().

```c
#include <stdio.h>
int get_int(char* prompt);
int isPerfect(int a);

int main(void)
{
    int L = get_int("Enter the lower limit L: ");
    int R = get_int("Enter the upper limit R: ");
    int counter = 0;
    for(int i = L; i<R+1; i++)
    {

        if(isPerfect(i))
        {
            counter++;
            printf("%d\t", i);
        }
    }
    printf("\n");
    printf("The number of perfect numbers in the given range including both the upper limit and lower limit is %d\n", counter);
    if (counter==0) printf("There is no a perfect number in the given range(including both L and R)\n");
    return 0;
}
int get_int(char* prompt)
{
    int a;
    printf("%s", prompt);
    scanf("%d", &a);
    return a;
}
int isPerfect(int a)
{
    // a perfect number is one whose perfect factors add up to the number itself
    int sum = 0;
    for(int i = 1 ; i<a;i++)
    {
        if(a%i == 0)
        {
            sum+=i;
        }
    }
    if(sum == a)
    {
        return 1;
```

```
    }
    return 0;
}
```

Output:

```
Enter the lower limit L: 1
Enter the upper limit R: 500
6       28      496
The number of perfect numbers in the given range including both the
```

14.Given a positive integer N, repeatedly perform the following operation until only one digit remains: replace N by the sum of its digits. Print the final single digit.
Input: An integer N.
Output: Print the final single digit.
Example:
Input: 9875
Output: 2

```
#include <stdio.h>
int get_int(char* prompt);
int dcount(int a);
int main(void)
{
    int N = get_int("Enter the value of N: ");
    while(dcount(N) != 1)
    {
        int copy = N;
        int sum = 0;
        while(copy>0)
        {
            sum+=(copy%10);
            copy/=10;
        }
        N =sum;
    }
    printf("The final single digit is : %d\n",N);
    return 0;
}
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
}
int dcount(int a)
{   //counting the number of digits
    if(a == 0)
    {
```

```
        return 1;
    }
    int count = 0;
    while(a>0)
    {
        a/=10;
        count++;
    }
    return count;
}
```

Output:

```
Enter the value of N: 64
The final single digit is : 1

Enter the value of N: 9875
The final single digit is : 2
```

15. Given an integer N, print the first N terms of a sequence generated as follows: start with 1. For
every subsequent term, multiply the previous term by its position and add the position. Use a loop and
display all terms in one line separated by spaces.
Input:
An integer N (1 ≤ N ≤ 20).
Output:
Print the N generated terms separated by spaces.
Example:
Input: 5
Output:
1 4 15 64 325
Write the program using appropriate loops and, where useful, a user-defined function.

```c
#include <stdio.h>
int get_int(char* prompt);
void printSequence(int N);

int main(void)
{
    int N = get_int("Enter the value of N: ");
    printSequence(N);
    return 0;
}
int get_int(char* prompt)
{
    printf("%s", prompt);
    int a;
    scanf("%d",&a);
    return a;
}
void printSequence(int N)
{
    // if 0 is entered give the sequence to be non existant because there will be 0 elements
    if(N==0)
    {
        printf("No Sequence for 0 elements\n");
        return;
    }
    // giving logic for the asked sequence
    printf("Following is the sequence: - \n");
    int element = 1;
    printf("%d ",element);
    for(int i = 2; i<=N; i++)
    {

        element = (element*i)+i;
        printf("%d ", element);
    }
    printf("\n");
    return;
}
```

Output

```
Enter the value of N: 5
Following is the sequence: -
1 4 15 64 325

Enter the value of N: 8
Following is the sequence: -
1 4 15 64 325 1956 13699 109600
```

Submitted by : Harshit Kakkar
Roll Number : 12611043
Branch : CSE A
Email :
harshitcse12611043@iiitsonepat.ac.in

Submitted on 22 October 2026
Course : BTech 2026-2030

Submitted To: Dr. Gourav Jain